

Chapter 9 explain

شرح السلايدات : (1 - 19)

Step 1: Mapping of Regular Entity Types

الخطوة 1: تحويل أنواع الكيانات العادية (القوية)

الشرح المبسط:

كل كيان قوي (Strong Entity) في مخطط ER يُحوَّل إلى جدول (Relation) واحد في قاعدة البيانات العلائقية. كل سمة بسيطة في الكيان تصبح عمودًا في الجدول، وأحد مفاتيحه الأساسية (Key) يصبح المفتاح الأساسي للجدول.

التفاصيل والعمق:

- يتم إنشاء علاقة (relation) باسم الكيان (مثل EMPLOYEE, DEPARTMENT, PROJECT).
- جميع السمات البسيطة (Simple Attributes) للكيان تُنقل مباشرة إلى هذا الجدول.
- إذا كانت السمة الرئيسية (Primary Key) مركبة (Composite)، فتُحوَّل إلى مجموعة من الأعمدة التي تمثل مكوناتها البسيطة، وتُصبح معًا المفتاح الأساسي للجدول.
- لا تُضاف أي سمات من علاقات أخرى هنا — فقط سمات الكيان نفسه.

مثال من الشريحة:

في مخطط COMPANY:

- EMPLOYEE → جدول به الأعمدة: SSN, FNAME, LNAME, BDATE, ADDRESS, SEX, SALARY, SUPERSSN, DNO
- DEPARTMENT → جدول: DNUMBER, DNAME, MGRSSN, MGRSTARTDATE → المفتاح: DNUMBER
- PROJECT → جدول: PNUMBER, PNAME, PLOCATION, DNUM → المفتاح: PNUMBER

ملاحظات مهمة:

- لا توجد سمات مشتقة (Derived) أو متعددة القيم (Multivalued) هنا — هذه تُعالج في خطوات لاحقة.
- الكيان القوي هو الذي يمكنه الوجود بشكل مستقل دون الحاجة إلى كيان آخر (عكس الكيان الضعيف).
- اختيار المفتاح لا يتغير — هو نفس المفتاح في ER.
- الهدف: الحفاظ على جميع السمات والمعلومات الأصلية (Goal: Preserve all information).

Step 2: Mapping of Weak Entity Types

الخطوة 2: تحويل أنواع الكيانات الضعيفة

الشرح المبسط:

الكيان الضعيف (Weak Entity) لا يمكنه الوجود بدون كيان مالك (Owner). لذلك، ندمجه مع جدول المالك، لكننا نخلق جدولاً خاصاً له، ونضيف مفتاح المالك كمفتاح أجنبي (Foreign Key)، ونستخدم المفتاح الجزئي (Partial Key) مع مفتاح المالك كمفتاح أساسي مركب.

التفاصيل والعمق:

- ننشئ جدولاً جديداً باسم الكيان الضعيف (مثل DEPENDENT).
- نضيف جميع السمات البسيطة للكيان الضعيف (مثل DEPENDENT_NAME, BDATE, SEX, RELATIONSHIP).
- نضيف المفتاح الأساسي للكيان المالك ك مفتاح أجنبي في هذا الجدول (مثل SSN من EMPLOYEE → يُعاد تسميته إلى ESSN).
- المفتاح الأساسي للجدول الجديد هو الجمع بين المفتاح الأجنبي (من المالك) + المفتاح الجزئي (Partial Key) للكيان الضعيف.
- لا يمكن أن يكون للمفتاح الأساسي للجدول قيمة NULL، لأنه مركب من مفتاحين غير قابلين للفراغ.

مثال من الشريحة:

- DEPENDENT ← جدول: ESSN, DEPENDENT_NAME, BDATE, SEX, RELATIONSHIP
- المفتاح الأساسي: {ESSN, DEPENDENT_NAME}
- EMPLOYEE(SSN) هو مفتاح أجنبي يشير إلى ESSN
- DEPENDENT_NAME هو المفتاح الجزئي (Partial Key) للكيان الضعيف

ملاحظات مهمة:

- الكيان الضعيف لا يملك مفتاحاً خاصاً به — فقط عبر الكيان المالك.
- إذا كان هناك أكثر من كيان مالك (نادر جداً)، فنضيف مفاتيحهم جميعاً كأجبيين.
- تسمية المفتاح الأجنبي غالباً ما تُغيّر لتوضيح المصدر (SSN → ESSN = Employee SSN).
- الهدف: الحفاظ على العلاقة التبعية (Dependency) والقيود (Constraints) بين الكيان الضعيف ومالكه.

Step 3: Mapping of Binary 1:1 Relationship Types

الخطوة 3: تحويل العلاقات الثنائية 1:1

الشرح المبسط:

عندما يكون هناك علاقة 1:1 بين كيانين (مثلاً: مدير واحد لكل إدارة، وإدارة واحدة لمدير واحد)، لدينا 3 طرق لتحويلها:

التفاصيل والعمق (الثلاث خيارات):

الخيار 1: Foreign Key Approach (2 جداول)

- نختار أحد الجدولين (S) ونضيف مفتاحه الأساسي من الجدول الآخر (T) كمفتاح أجنبي فيه.
- **الأفضل:** اختيار الكيان الذي لديه مشاركة كاملة (Total Participation) في العلاقة.
- لماذا؟ لأن ذلك يضمن عدم وجود قيم NULL، ويجعل الاستعلامات أكثر كفاءة.

مثال: العلاقة MANAGES بين EMPLOYEE و DEPARTMENT

- DEPARTMENT (كل إدارة لديها مدير) لها مشاركة كاملة
- EMPLOYEE (قد لا يكون مديرًا (مشاركة جزئية) EMPLOYEE(SSN) يشير إلى DEPARTMENT كمفتاح أجنبي في جدول MGRSSN إذاً نضيف →
• نحذف العلاقة ككيان منفصل — فهي الآن جزء من جدول الإدارة.

الخيار 2: Merged Relation (1 جدول)

- ندمج كلا الكيانيين والعلاقة في جدول واحد.
- يُستخدم فقط عندما تكون المشاركتان كاملتان (Total Participation) لكلا الكيانيين.
- مثال: إذا كان كل موظف هو مدير لأحد الإدارات، وكل إدارة لها مدير واحد فقط — ندمج كل سماتهما في جدول واحد.
- نحذف المفتاح الأجنبي لأنه أصبح جزءًا من نفس الجدول.

الخيار 3: Cross-reference / Relationship Relation (3 جداول)

- ننشئ جدولًا منفصلًا للعلاقة (مثل MANAGES) يحتوي فقط على مفتاحين أجنبيين:
- DNUMBER (من DEPARTMENT)
- SSN (من EMPLOYEE)
- هذا الخيار نادر الاستخدام لأنه يزيد التعقيد ويستهلك مساحة غير ضرورية.
- قد يُستخدم فقط إذا كانت العلاقة تحتوي على سمات خاصة (مثلًا: DateOfAppointment) — لكن حتى هنا، الأفضل دمجها في أحد الجدولين.

ملاحظات مهمة:

- الخيار 1 هو الأكثر شيوعًا وفعالية في التطبيقات العملية.
- **الهدف:** تقليل عدد الجداول وتجنب NULLs قدر الإمكان.
- لا ننسى: في ER، العلاقة 1:1 لا تحتوي على سمات إلا إذا كانت "وصفية" — وإذا كانت كذلك، تُضاف في الجدول الذي يحتوي على المفتاح الأجنبي (الخيار 1) أو في الجدول المدمج (الخيار 2).

Step 4: Mapping of Binary 1:N Relationship Types

الخطوة 4: تحويل العلاقات الثنائية N:1

الشرح المبسط:

العلاقة 1:N (واحد إلى كثير) — مثل: إدارة واحدة لها العديد من الموظفين — تُحول بإضافة المفتاح الأساسي للجانب "1" كمفتاح أجنبي في جدول الجانب "N".

التفاصيل والعمق:

- نحدد الجدول الذي يمثل الكيان على جانب "N" (الذي يحتوي على الكثير من الكيانات).
- نضيف المفتاح الأساسي للكيان على جانب "1" ك مفتاح أجنبي داخل جدول "N".
- أي سمات بسيطة للعلاقة نفسها (مثلاً: StartDate في WORKS_FOR) تُضاف كأعمدة في جدول "N".
- لا ننشئ جدولاً منفصلاً للعلاقة — ندمجها في جدول "N".

مثال من الشريحة:

- العلاقة WORKS_FOR بين (1) DEPARTMENT و (N) EMPLOYEE → نضيف DNUMBER (من DEPARTMENT) كمفتاح أجنبي في جدول EMPLOYEE → نسميه DNO
- العلاقة CONTROLS بين (1) DEPARTMENT و (N) PROJECT → نضيف DNUM كمفتاح أجنبي في جدول PROJECT
- العلاقة SUPERVISION بين (1) EMPLOYEE و (N) EMPLOYEE — علاقة ذاتية → نضيف SUPERSSN كمفتاح أجنبي في جدول EMPLOYEE (يشير إلى SSN)

ملاحظات مهمة:

- لا يوجد جدول منفصل للعلاقة — هذا يختلف تمامًا عن M:N.
- السمات الخاصة بالعلاقة (مثل HOURS في WORKS_ON) لا تُضاف هنا — لأنها ليست في علاقة 1:N، بل في M:N.
- الاستثناء الوحيد: إذا كانت العلاقة 1:N تحتوي على سمات، فهي تُضاف دائمًا إلى جانب "N".
- السبب المنطقي: كل كيان في الجانب "N" يحتاج لمعرفة من هو "الطرف 1" الذي يرتبط به — لذا نضع المفتاح هناك.
- الهدف: تقليل الجداول، تجنب الجداول الزائدة، الحفاظ على كفاءة الاستعلامات.

Step 5: Mapping of Binary M:N Relationship Types

الخطوة 5: تحويل العلاقات الثنائية N:M

الشرح المبسط:

العلاقات N:M (كثير إلى كثير) — مثل: موظف يعمل على عدة مشاريع، ومشروع له عدة موظفين — لا يمكن تمثيلها ببساطة بالمفتاح الأجنبي. لذا ننشئ جدولاً منفصلاً للعلاقة.

التفاصيل والعمق:

- ننشئ جدولاً جديداً باسم العلاقة (مثل WORKS_ON).
- نضيف كل مفتاح أساسي من الجدولين المشاركين ك مفاتيح أجنبية في هذا الجدول.
- المفتاح الأساسي للجدول الجديد هو الجمع بين هاتين المفتاحين الأجنبيين.
- أي سمات بسيطة للعلاقة (مثل HOURS) تُضاف كأعمدة في هذا الجدول.

- لا تُضاف أي سمات من الكيانات الأصلية — فقط مفاتيحها وسمات العلاقة.

مثال من الشريحة:

- العلاقة WORKS_ON بين EMPLOYEE و PROJECT

→ جدول: ESSN , PNO , HOURS

- EMPLOYEE(SSN) مفتاح أجنبي من ← ESSN
- PROJECT(PNUMBER) مفتاح أجنبي من ← PNO

- المفتاح الأساسي: {ESSN, PNO}

- HOURS ← سمة للعلاقة نفسها

ملاحظات مهمة:

- هذا هو الحل الوحيد الصحيح للعلاقات N:M.
- لا يمكن استخدام المفتاح الأجنبي في أحد الجدولين — لأنه سيؤدي إلى تكرار أو فقدان معلومات.
- الجدول الناتج يمثل جدول ربط (Join Table).
- إذا كانت العلاقة تحتوي على سمة مركبة (Composite Attribute)، نفكها إلى مكوناتها البسيطة.
- الهدف: تمثيل العلاقة بدقة، مع الحفاظ على جميع السمات، وتجنب التكرار أو فقدان البيانات.

Step 6: Mapping of Multivalued Attributes

الخطوة 6: تحويل السمات المتعددة القيم

الشرح المبسط:

إذا كان لدى كيان سمة يمكن أن تحتوي على قيم متعددة (مثلاً: مكان عمل واحد للموظف؟ لا — الموظف قد يعمل في عدة أماكن!) — ننشئ جدولاً منفصلاً لهذه السمة.

التفاصيل والعمق:

- ننشئ جدولاً جديداً باسم السمة (مثل DEPT_LOCATIONS).
- نضيف كعمود: القيمة الفعلية للسمة المتعددة (مثل DLOCATION).
- نضيف كعمود: المفتاح الأساسي للكيان أو العلاقة التي تنتمي إليه السمة كمفتاح أجنبي.
- المفتاح الأساسي للجدول الجديد هو الجمع بين المفتاح الأجنبي + السمة المتعددة القيم.
- إذا كانت السمة مركبة (مثل Address: Street, City, Zip)، نفكها إلى أعمدة منفصلة.

مثال من الشريحة:

- السمة LOCATIONS في DEPARTMENT (عدد غير محدود من المواقع)

→ جدول: DNUMBER , DLOCATION

- DEPARTMENT(DNUMBER) مفتاح أجنبي من ← DNUMBER
- DLOCATION ← القيمة المتعددة

- المفتاح الأساسي: {DNUMBER, DLOCATION}

ملاحظات مهمة:

- السمات المتعددة القيم غير مقبولة في النموذج العلائقي — لذا يجب تفكيكها.
- هذا التحويل يحافظ على الطبيعة "متعددة القيم" دون انتهاك القواعد العلائقية (1NF).
- لا تُستخدم هذه الطريقة للعلاقات — فقط للسمات.
- إذا كانت السمة المتعددة تخص علاقة (ليس كياناً)، نفس المبدأ يطبق.
- الهدف: الحفاظ على المعلومات الكاملة دون تكرار أو فقدان — مع الالتزام بقواعد 1NF.

Step 7: Mapping of N-ary Relationship Types

الخطوة 7: تحويل العلاقات N-ary (أكثر من ثنائي)

الشرح المبسط:

العلاقات التي تربط أكثر من كيانين (مثل 3 أو 4 كيانات) — مثل: مورد يزود جزءاً لمشروع — لا يمكن تمثيلها بمفتاح أجنبي بسيط. ننشئ جدولاً منفصلاً للعلاقة.

التفاصيل والعمق:

- ننشئ جدولاً جديداً باسم العلاقة (مثل SUPPLY).
- نضيف كل مفتاح أساسي من كل كيان مشارك ك مفتاح أجنبي في هذا الجدول.
- المفتاح الأساسي للجدول الجديد هو الجمع بين كل المفاتيح الأجنبية.
- أي سمات بسيطة للعلاقة (مثل Quantity, Date) تُضاف كأعمدة.

مثال من الشريحة (الشكل 9.2 & Figure 3.17):

- العلاقة SUPPLY بين: SUPPLIER (SNAME), PART (PARTNO), PROJECT (PROJNAME)
→ جدول: SNAME , PARTNO , PROJNAME , QUANTITY
- SUPPLIER مفتاح أجنبي من ← SNAME
- PART مفتاح أجنبي من ← PARTNO
- PROJECT مفتاح أجنبي من ← PROJNAME
- المفتاح الأساسي: {SNAME, PARTNO, PROJNAME}
- QUANTITY ← سمة للعلاقة

ملاحظات مهمة:

- لا توجد طريقة أخرى — لا يمكن دمجها في جداول الكيانات.
- هذا الجدول يمثل نقطة تقاطع ثلاثية (Ternary Junction).
- إذا كان أحد الكيانات له مشاركة جزئية — لا توجد مشكلة، ستظهر قيم NULL (لكن في ER لا توجد NULLات في العلاقات، لذا يجب التحقق من التصميم).
- الهدف: تمثيل العلاقة المعقدة بدقة، مع الحفاظ على جميع المعلومات والقيود.

Mapping EER Model Constructs to Relations

تحويل مكونات نموذج EER إلى علاقات

مقدمة:

و الأنواع المتصلة (Generalization) و التصنيف العام (Specialization) يضيف مفاهيم جديدة: التخصص (Union Types/Categories).

هذه الخطوات (8 و 9) تتعاملان معها.

Step 8: Options for Mapping Specialization or Generalization

الخطوة 8: خيارات تحويل التخصص أو التعميم

الشرح المبسط:

عندما يكون لدينا كيان عام (Superclass) وعدد من الكيانات المتخصصة (Subclasses) — مثل: EMPLOYEE → SECRETARY, ENGINEER, MANAGER — كيف نحولها؟ لدينا 4 خيارات.

التفاصيل والعمق (الخيارات الأربعة):

Option 8A: Multiple relations — Superclass and subclasses

(علاقته متعددة — الكيان العام والفرعي)

- ننشئ جدولاً للكيان العام (C):
 - $Attrs(C) = \{k, a_1, \dots, a_n\} \rightarrow$ المفتاح k
- ننشئ جدولاً لكل فئة فرعية (Si):
 - $Attrs(S_i) = \{k\} \cup \{attributes\ of\ S_i\}$
 - $PK(S_i) = k$ (نفس مفتاح الكيان العام)
 - هو مفتاح أساسي في كل جدول، وهو أيضاً مفتاح أجنبي في الجداول الفرعية يشير إلى الكيان العام k .

مثال:

- EMPLOYEE (SSN, FNAME, LNAME, BDATE, ADDRESS, SEX, SALARY)
- SECRETARY (SSN, TypingSpeed)
- ENGINEER (SSN, Degree, Area)
- MANAGER (SSN, DeptManaged, Level)

• كل جدول فرعي يحتوي على SSN كمفتاح أساسي ومفتاح أجنبي لـ EMPLOYEE

مميزات: 

- يدعم أي نوع تخصص: كلي (total) أو جزئي (partial)، منفصل (disjoint) أو متقاطع (overlapping).

- يحافظ على البنية الأصلية بدقة.
- يسمح بإضافة كيانات فرعية لاحقاً دون تغيير الجدول العام.

❌ عيوب:

- قد يؤدي إلى تكرار المفتاح الأساسي في عدة جداول.
- استعلامات "جميع الموظفين" تتطلب JOIN بين الجداول.

Option 8B: Multiple relations — Subclass relations only

(علاقتان متعددتان — فقط الجداول الفرعية)

- ننشئ جدولاً فقط لكل فئة فرعية (Si).
- $Attrs(Si) = \{all\ attributes\ of\ Si\} \cup \{all\ attributes\ of\ C\}$
- $PK(Si) = k$ (مفتاح الكيان العام)

مثال:

- SECRETARY (SSN, FNAME, LNAME, BDATE, ADDRESS, SEX, SALARY, TypingSpeed)
- ENGINEER (SSN, FNAME, LNAME, BDATE, ADDRESS, SEX, SALARY, Degree, Area)

✓ مميزات:

- لا حاجة لجدول عام — تبسيط في بعض الاستعلامات.
- لا JOIN مطلوب عند السؤال عن نوع معين.

❌ شروط العمل فقط:

- يجب أن تكون المشاركة كاملة (Total Participation) — أي كل كيان في C يجب أن يكون في أحد subclasses.
- لا يمكن استخدامه إذا كان التخصص جزئياً (Partial) — لأن بعض الموظفين لن يُدرجوا!
- لا يمكن استخدامه إذا كان التخصص متقاطعاً (Overlapping) — لأن الموظف قد يكون مهندساً ومديرًا في نفس الوقت — ولكن الجدول لا يسمح بذلك.

Option 8C: Single relation with one type attribute

(جدول واحد مع سمة نوع واحدة)

- ننشئ جدولاً واحداً يضم جميع سمات الكيان العام + جميع سمات الفروع.
- نضيف سمة واحدة فقط: Type (مثلاً: 'Type' = 'Secretary', 'Engineer', 'Manager')
- السمات غير المستخدمة في سطر معين تُترك ك NULL.

مثال:

- EMPLOYEE (SSN, FNAME, LNAME, BDATE, ADDRESS, SEX, SALARY, TypingSpeed, Degree, Area, DeptManaged, Level, Type)
- موظف مكتب: TypingSpeed ≠ NULL، باقي السمات = NULL

✓ مميزات:

- جدول واحد فقط — بساطة في الاستعلامات.
- يدعم التخصيص الجزئي والمتداخل.

✗ عيوب:

- كثير من القيم NULL — يخالف هدف "تقليل القيم الفارغة".
- صعوبة في التحقق من القيود (مثلاً: هل المهندس لديه Degree؟ لا يمكن التحقق بسهولة).
- استهلاك مساحة غير فعال.

Option 8D: Single relation with multiple type attributes

(جدول واحد مع سمات نوع متعددة)

- جدول واحد يضم كل السمات.
- لكن بدلاً من سمة واحدة "Type"، نضيف سمات منطقية (Boolean) لكل فئة فرعية.
- مثلاً: IsSecretary, IsEngineer, IsManager — كلها من نوع BOOLEAN (0/1)

مثال:

- EMPLOYEE (SSN, FNAME, LNAME, ..., TypingSpeed, Degree, Area, DeptManaged, Level, IsSecretary, IsEngineer, IsManager)
- موظف مهندس: IsEngineer=1, IsSecretary=0, IsManager=0

✓ مميزات:

- يدعم التخصيص الجزئي والمتداخل.
- لا توجد قيم NULL في السمات الخاصة بالفئات — فقط في السمات العامة.

✗ عيوب:

- القيود غير مضمونة تلقائياً — يمكن أن يكون IsSecretary=1 و IsEngineer=1 (متداخل) — وهذا قد يكون مقبولاً، لكن قد لا يكون.
- يجب إضافة قيود (Constraints) يدوياً في قاعدة البيانات لمنع التراكب غير المسموح به.
- تعقيد في التحقق من التخصيص المنفصل (Disjoint).

ملاحظات مهمة:

الخيار	يدعم ?Total	يدعم ?Partial	يدعم ?Disjoint	يدعم ?Overlapping	القيم NULL؟	الأفضل لـ
8A	✓	✓	✓	✓	✗ (نادرة)	التصميم الدقيق، الأمثلة الأكاديمية
8B	✓	✗	✓	✗	✗	التخصص الكامل فقط
8C	✓	✓	✓	✓	✓ (كثير)	تبسيط الاستعلامات
8D	✓	✓	⚠ (بحاجة لقيود)	✓	✗ (في سمات الخاصة)	التخصص المتداخل

📌 ملاحظة حاسمة:

في الاختبارات، غالبًا ما يُطلب منك اختيار الخيار المناسب بناءً على نوع التخصص (total/partial, disjoint/overlapping).

- إذا كان التخصص جزئيًا أو متداخلًا → الخيار 8A أو 8C أو 8D.
- إذا كان كليًا ومنفصلًا → يمكنك استخدام 8B (الأكثر كفاءة).
- إذا طُلب تقليل 8A → NULLs أو 8B أو 8D.
- إذا طُلب تبسيط الجداول → 8C أو 8D.

Step 9: Mapping of Union Types (Categories)

الخطوة 9: تحويل أنواع الاتحاد (الفئات)

الشرح المبسط:

نوع الاتحاد (Category) هو كيان يجمع كيانات من أنواع مختلفة (ليس من تخصص واحد).
مثال: `EMPLOYEE_OR_CONTRACTOR` — يمكن أن يكون موظفًا أو متعاقدًا — وكلاهما من كيانات مختلفة!

التفاصيل والعمق:

- لا يمكن تمثيلها كتخصص (specialization) لأنها تجمع كيانات من مجموعات مختلفة (ليس فروعًا لنفس الكيان).
- الحل: نستخدم خيارين فقط:

1. استخدام Option 8A مع تغيير بسيط:

- ننشئ جدولًا للنوع العام (Category) — لكنه لا يحتوي على أي سمات.
- ننشئ جداول منفصلة لكل كيان مشارك (Employee, Contractor) — وكل جدول يحتوي على المفتاح الأساسي للنوع العام كمفتاح أجنبي.
- لكن: هذا لا يضمن أن الكيان موجود في واحد فقط — لأنه لا توجد علاقة تبعية.

2. استخدام Option 8C أو 8D:

- ننشئ جدولًا واحدًا يحتوي على سمات جميع الكيانات الممكنة.
- نضيف سمة "Type" أو سمات منطقية (IsEmployee, IsContractor).

- نصيف قيودًا لضمان أن أحد القيم فقط هو 1 (مثل IsEmployee + CHECK constraint: IsContractor = 1).

مثال من الشريحة:

- Category: EMPLOYEE_OR_CONTRACTOR
- يشمل: EMPLOYEE (SSN, Name, Salary) و CONTRACTOR (ID, Name, HourlyRate)
- الحل الأمثل: جدول واحد:
- Person (SSN, ID, Name, Salary, HourlyRate, Type)
- Type = 'E' أو 'C'
- قيد: (SSN IS NULL AND ID IS NOT NULL) OR (SSN IS NOT NULL AND ID IS NULL)

ملاحظات مهمة:

- الاتحاد (Category) ≠ التخصص (Specialization)
- التخصص: فروع من نفس الكيان (مثل موظف → مدير/مهندس).
- الاتحاد: كيانات من أصول مختلفة (موظف أو متعاقد).
- لا يوجد حل مثالي — كل الحلول فيها تنازلات.
- في التصميم العملي، غالبًا نستخدم خيار 8C أو 8D لتبسيط الوصول.
- الهدف: تمثيل كيانات تأتي من مصادر مختلفة ضمن كيان واحد — مع الحفاظ على التكامل.

ملخص نهائي: Summary of Mapping constructs and constraints

(ملخص تحويل المكونات والقيود)

الملاحظات	التحويل في العلائقي	المكون في ER/EER
لا توجد NULLs في المفتاح	جدول واحد، سماته أعمدة، المفتاح الأساسي هو المفتاح	Entity Type (Regular)
لا يمكن وجوده بدون المالك	جدول منفصل، مع مفتاح أجنبي من المالك + المفتاح الجزئي كمفتاح أساسي مركب	Weak Entity
تجنب الجدول المنفصل	مفتاح أجنبي في أحد الجدولين (الأفضل في ذي المشاركة الكاملة) أو دمج أو جدول منفصل	1:1 Relationship
لا جدول منفصل	مفتاح أجنبي في جدول الجانب N + سمات العلاقة إن وجدت	1:N Relationship
المفتاح الأساسي هو الجمع بين المفتاحين	جدول منفصل يحتوي على مفاتيح أجنبية من كلا الكيانين + سمات العلاقة	M:N Relationship

الملاحظات	التحويل في العلائقي	المكون في ER/EER
يحل مشكلة 1NF	جدول منفصل: مفتاح أجنبي + السمة المتعددة كمفتاح أساسي مركب	Multivalued Attribute
المفتاح الأساسي هو الجمع بين كل المفاتيح	جدول منفصل: مفاتيح أجنبية من كل كيان مشارك + سمات العلاقة	N-ary Relationship (n>2)
اختر حسب نوع التخصص (Total/Partial,) (Disjoint/Overlapping)	4 خيارات: 8A (الأكثر أماناً)، 8B (لكن فقط إذا كان كلياً)، 8C (مع NULLs)، 8D (مع سمات منطقية)	Specialization (Generalization)
لا يوجد حل مثالي — غالباً 8C أو 8D	جدول واحد مع سمات من جميع الأنواع + سمة نوع أو قيود منطقية	Union Type (Category)

🔍 ملاحظات عامة شاملة (Important Notes for Exam)

1. الهدف الرئيسي من التحويل:

- ✓ الحفاظ على جميع السمات والمعلومات
- ✓ الحفاظ على القيود قدر الإمكان (لكن بعضها غير ممكن في العلائقي: مثل نسبة 1:10 أو التصنيف الكامل)
- ✓ تقليل القيم الفارغة (NULLs) قدر الإمكان

2. القيود التي لا يمكن الحفاظ عليها في النموذج العلائقي:

- النسبة القصوى (Max Cardinality Ratio) مثل 1:10 — لا يمكن تطبيقها تلقائياً.
- التصنيف الكامل (Exhaustive Classification) — مثل: "كل موظف هو إما مهندس أو مدير" — لا يمكن التحقق منه تلقائياً إلا ببرمجة أو قيود مخصصة.
- التخصص المنفصل (Disjoint) — في الخيارات 8C و 8D، يجب تطبيق قيود يدوية.

3. الاختيارات المثلى في التصميم:

- دائماً اختر الخيار 8A إذا كنت غير متأكد — فهو الأكثر أماناً وشمولاً.
- استخدم الخيار 8B فقط إذا طُلب التخصص الكامل والمنفصل.
- تجنب الجداول المنفصلة للعلاقات 1:1 و N:1 إلا إذا طُلب صراحة.
- لا تنسَ: كل علاقة M:N و multivalued و n-ary → جدول منفصل.

4. تسمية المفاتيح الأجنبية:

- دائماً أعد تسميتها لتوضيح المصدر (مثل SSN → ESSN, DNUMBER → DNO).
- هذا يساعد على قراءة الجداول وفهم العلاقات.

5. الفرق بين التخصص والاتحاد:

- Specialization:** "الموظف هو نوع من..." ← كيان واحد ينقسم.
- Category:** "الشخص هو موظف أو متقاعد" ← كيانان مختلفان يندمجان.

📌 خلاصة سريعة للحفظ (Cheat Sheet)

المفهوم	التحويل	مفتاح أساسي	ملاحظة
كيان قوي	جدول	سمة المفتاح الأصلي	لا تغيير
كيان ضعيف	جدول + مفتاح أجنبي من المالك	{مفتاح المالك + المفتاح الجزئي}	لا يمكنه الوجود بدون المالك
علاقة 1:1	مفتاح أجنبي في أحد الجدولين (الأفضل في ذي المشاركة الكاملة)	كما في الجدول المضيف	تجنب الجدول الثالث
علاقة N:1	مفتاح أجنبي في جدول N	كما في جدول N	لا جدول منفصل
علاقة M:N	جدول منفصل	{مفتاح A + مفتاح B}	لا بديل
سمة متعددة القيم	جدول منفصل	{مفتاح الكيان + القيمة}	حل لـ 1NF
علاقة n-ary	جدول منفصل	{مفتاح كل كيان مشارك}	مثال: SUPPLY (S,P,J)
تخصص (Specialization)	4 خيارات: 8A (آمن)، 8B (كلي فقط)، 8C (NULLs) (منطقية)، 8D	في كل خيار: المفتاح الأساسي هو مفتاح الكيان العام	اختر حسب نوع التخصص
نوع اتحاد (Category)	جدول واحد مع سمات من جميع الأنواع + نوع أو قيود	يعتمد على التصميم	غالبًا 8C أو 8D

شرح السلايدات : (20 - 38)

Mapping EER Model Constructs to Relations (contd.)

ترجمة: تحويل مفاهيم نموذج EER إلى علاقات (مستمر)

Option 8C: Single relation with one type attribute

ترجمة: علاقة واحدة مع سمة نوع واحدة

- الوصف:

عند وجود تخصص (Specialization) أو تعميم (Generalization)، يمكننا تمثيل جميع الكلاسات (السوبر كلاس والسوبر كلاسات الفرعية) في علاقة واحدة فقط (relation). هذه العلاقة تحتوي على:

- جميع السمات الأساسية للسوبر كلاس (مثل: $(k, a_1, \dots, a_n)$ — حيث k هو المفتاح الأساسي (Primary Key).
- جميع السمات الخاصة بكل فئة فرعية $(S_1, S_2, \dots, S_m)$.
- سمة إضافية واحدة تسمى **Type Attribute (t)** — وهي سمة تُستخدم لتحديد أي فئة فرعية ينتمي كل تسجيل (tuple).

- المفتاح الأساسي:

أي أن المفتاح الأساسي للعلاقة الجديدة هو نفس مفتاح السوبر كلاس الأصلي $PK(L) = k$.

- **مثال عملي:**

إذا كان لدينا سوبر كلاس EMPLOYEE مع فئتين فرعيتين: ENGINEER و MANAGER ، فإن العلاقة الناتجة ستكون:

EMPLOYEE (emp_id, name, salary, job_title, department, type)

حيث type قد تكون قيمتها "Engineer" أو "Manager".

- **المزايا:**

- بساطة التصميم: علاقة واحدة فقط.
- سهولة الاستعلام على جميع الموظفين بغض النظر عن نوعهم.

- **العيوب:**

- وجود العديد من السمات التي قد تكون NULL لمعظم السجلات (مثلاً: سمة "car_model" ستكون NULL لكل المهندسين إذا كانت خاصة بالمديرين فقط).
- صعوبة ضمان التحقق من صحة البيانات (مثلاً: يجب التأكد أن قيمة "type="Engineer" تعني أن جميع سمات ENGINEER غير فارغة).
- لا تدعم التخصصات المتداخلة (Overlapping) بشكل طبيعي.

- **ملاحظة مهمة:**

هذا الخيار مناسب جداً عندما تكون الفئات الفرعية منفصلة (disjoint)، ولا يوجد تداخل بينها.

Option 8D: Single relation with multiple type attributes

ترجمة: علاقة واحدة مع عدة سمات نوع

- **الوصف:**

بدلاً من استخدام سمة واحدة (type) لتحديد الفئة، نستخدم سمات منطقية (Boolean) — كل سمة تمثل فئة فرعية واحدة.

العلاقة تحتوي على:

- سمات السوبر كلاس الأصلية (k, a1, ..., an).
- جميع السمات الخاصة بكل فئة فرعية (S1, S2, ..., Sm).
- سمات نوع متعددة: t1, t2, ..., tm — حيث ti هي سمة من نوع BOOLEAN (صحيح/خطأ) تشير إلى ما إذا كان السجل ينتمي إلى الفئة الفرعية Si.

- **المفتاح الأساسي:**

نفس مفتاح السوبر كلاس $PK(L) = k \rightarrow$

- **مثال عملي:**

لنفترض أن لدينا سوبر كلاس VEHICLE مع فئتين فرعيتين: CAR و TRUCK . العلاقة الناتجة ستكون:

VEHICLE (vin, make, model, year, car_flag, truck_flag)

- إذا كان السجل سيارة: car_flag = TRUE , truck_flag = FALSE
- إذا كان الشاحنة: car_flag = FALSE , truck_flag = TRUE
- إذا كان التخصص متداخلاً (overlapping)، يمكن أن يكون كلاهما TRUE .

- **المزايا:**
- يدعم التخصص المتداخل (**Overlapping Specialization**) بشكل طبيعي — لأن أكثر من سمة نوع يمكن أن تكون صحيحة في نفس السجل.
- يسهل التحقق من الانتماءات المعقدة.
- **العيوب:**
- زيادة عدد السمات (إذا كان هناك 10 فئات فرعية، سيكون لدينا 10 أعمدة منطقية).
- الحاجة إلى قيود (constraints) لضمان أن على الأقل سمة واحدة صحيحة (في حالة تخصص غير متداخل)، أو أنه لا يمكن أن تكون جميعها خاطئة.
- قد يؤدي إلى تشويش في فهم البيانات بسبب كثرة الأعمدة المنطقية.
- **ملاحظة مهمة:**
- هذا الخيار هو الأفضل لحالات التخصص المتداخل (**Overlapping**)، لأنه يسمح لسجل واحد بالانتماء لأكثر من فئة فرعية.

FIGURE 4.4 EER diagram notation for an attribute-defined specialization on JobType.

ترجمة: الرمزية البيانية لنموذج EER للتخصص المعرف بالسمة JobType

- **الشرح:**
- هذه الشريحة توضح كيف يُظهر نموذج EER التخصص المحدد بالسمة (**Attribute-Defined Specialization**).
- في هذا المثال، السوبر كلاس هو EMPLOYEE ، والفئات الفرعية هي ENGINEER ، MANAGER ، SALESREP .
- التخصص مُعرّف بواسطة سمة واحدة في السوبر كلاس تسمى JobType (مثلاً: قيمها "Engineer", "Manager", "SalesRep").
- هذا يعني أن تصنيف الموظف يتم تلقائيًا بناءً على قيمة JobType — وليس عن طريق دخول بيانات منفصلة.
- **الربط مع الخيارات 8C و 8D:**
- هذا النوع من التخصص يتوافق تمامًا مع **Option 8C**، لأننا نستخدم سمة واحدة (JobType) لتحديد الفئة الفرعية. في التحويل إلى قاعدة بيانات علائقية، سيتم استخدام JobType كـ type attribute في العلاقة الوحيدة.
- **ملاحظة مهمة:**
- التخصص المعرف بالسمة (**Attribute-Defined**) يمنع التخصص المتداخل، لأن القيمة الواحدة للسمة لا يمكن أن تكون أكثر من قيمة في وقت واحد.
- لذلك، لا يمكن استخدام **Option 8D** هنا إلا إذا غيرنا طبيعة التخصص لتكون غير محددة بالسمة (**Value-Defined**).

Mapping the EER schema in Figure 4.4 using option 8A

ترجمة: تحويل مخطط EER في الشكل 4.4 باستخدام الخيار 8A

- **الشرح:**
- الخيار 8A هو: إنشاء علاقة منفصلة لكل فئة فرعية، حيث تُورث السمات الأساسية من السوبر كلاس إلى كل فئة فرعية.

- كل فئة فرعية لها علاقتها الخاصة (مثل: ENGINEER(emp_id, name, salary, job_type, dept, (engineering_skill)).

- المفتاح الأساسي لكل علاقة فرعية هو نفس مفتاح السوبر كلاس (emp_id) — وهو أيضًا مفتاح خارجي (Foreign Key) يشير إلى الجدول الأصلي EMPLOYEE.

- النتيجة:

- 3 علاقات: EMPLOYEE , ENGINEER , MANAGER , SALESREP

- تحتوي على السمات المشتركة EMPLOYEE.

- كل فئة فرعية تحتوي على السمات المشتركة + سماتها الخاصة.

- المزايا:

- تنظيم واضح: كل فئة لها جدول مستقل.

- لا توجد قيم NULL (ما لم تكن السمات الخاصة اختيارية).

- سهل الحفاظ على التكامل (Integrity) لأن كل نوع له هيكله الخاص.

- العيوب:

- يحتاج إلى عمليات JOIN عند استرجاع جميع الموظفين بغض النظر عن نوعهم.

- إذا أردنا حساب عدد الموظفين، نحتاج لدمج 3 جداول — مما يزيد التعقيد.

- ملاحظة مهمة:

الخيار 8A هو الأكثر استخدامًا في التطبيقات العملية لأنه يحافظ على التكامل ويسهل التوسع، لكنه أقل كفاءة في الاستعلامات العامة.

Mapping the EER schema in Figure 4.4 using option 8C

ترجمة: تحويل مخطط EER في الشكل 4.4 باستخدام الخيار 8C

- الشرح:

هنا، نستخدم علاقة واحدة فقط لتمثيل جميع الفئات.

- العلاقة: EMPLOYEE(emp_id, name, salary, job_type, dept, skill)

- السمة job_type هي **type attribute**، وقيمها: "Engineer", "Manager", "SalesRep".

- السمة skill قد تكون مشتركة أو مخصصة — لكن في هذا المثال، تُستخدم كسمة واحدة قد تحتوي على معلومات مختلفة حسب النوع.

- الفرق بينه وبين 8A:

في 8A: 4 جداول — في 8C: جدول واحد.

في 8A: لا توجد قيم NULL (إلا إذا كانت السمات اختيارية).

في 8C: معظم السمات ستكون NULL في معظم السجلات (مثلاً: مهارات المدير ستكون NULL للمهندس).

- ملاحظة مهمة:

هذا الخيار يتوافق مع التخصيص المعرف بالسمة (Attribute-Defined) — لأن القيمة job_type تحدد التصنيف تلقائيًا.

لا يمكن استخدامه إذا كان التخصيص غير معرف بالسمة (Value-Defined) أو متداخلًا.

FIGURE 4.3(b) Generalizing CAR and TRUCK into the superclass VEHICLE.

ترجمة: تعميم السيارة والشاحنة إلى السوبر كلاس VEHICLE

- الشرح:
هنا لدينا كيانان: CAR و TRUCK ، ويتم تعميمهما إلى سوبر كلاس جديد: VEHICLE .
 - VEHICLE : vin , make , model , year : يحتوي على السمات المشتركة مثل
 - CAR : num_doors : لديه سمة خاصة
 - TRUCK : cargo_capacity : لديه سمة خاصة
- الهدف:
تمثيل العلاقات المشتركة بطريقة منهجية — لتجنب التكرار.
- التطبيق العملي:
يمكن تطبيق أي من الخيارات 8A، 8B، 8C، 8D على هذا التعميم.

Mapping the EER schema in Figure 4.3b using option 8B.

ترجمة: تحويل مخطط EER في الشكل 4.3b باستخدام الخيار 8B

- الوصف:
الخيار 8B: إنشاء علاقة للسوبر كلاس + علاقة لكل فئة فرعية تحتوي فقط على السمات الخاصة بها.
- النتائج:
 - جدول VEHICLE(vin, make, model, year) — يحتوي فقط على السمات المشتركة.
 - جدول CAR(vin, num_doors) — vin هو مفتاح أساسي ومفتاح خارجي يشير إلى VEHICLE .
 - جدول TRUCK(vin, cargo_capacity) — vin هو مفتاح أساسي ومفتاح خارجي.
- المزايا:
 - لا تكرار للبيانات المشتركة.
 - توفير مساحة تخزين.
 - سهولة تحديث السمات المشتركة (في جدول واحد فقط).
- العيوب:
 - كل استعلام عن سيارة يتطلب JOIN بين VEHICLE و CAR .
 - صعوبة في التحقق من التبعية: يجب التأكد أن كل vin في CAR موجود في VEHICLE .
- ملاحظة مهمة:
هذا الخيار هو الأكثر كفاءة من حيث التخزين، ولكنه الأقل كفاءة من حيث الأداء بسبب الحاجة المستمرة لـ JOINS . يُستخدم غالبًا عندما تكون السمات المشتركة كثيرة والخصائص الفرعية قليلة.

FIGURE 4.5 An overlapping(non-disjoint) specialization.

ترجمة: تخصص متداخل (غير منفصل)

- الشرح:
التخصص المتداخل (Overlapping) يعني أن كائنًا واحدًا يمكن أن ينتمي إلى أكثر من فئة فرعية واحدة.
 - مثال: شخص يمكن أن يكون موظفًا ومالك سيارة في نفس الوقت.
 - مثال آخر: سيارة يمكن أن تكون سيارة رياضية AND سيارة كهربائية.
- الرمز البياني:
في EER، يُشار إلى التخصص المتداخل بـ خط متقطع أو بدون علامة "disjoint" — بينما التخصص المنفصل يُشار إليه بعلامة "d" داخل دائرة.
- التأثير على التحويل:
الخيارات 8A و 8B و 8C لا يمكنها تمثيل التخصص المتداخل بشكل صحيح — لأنها تفترض أن كل كائن ينتمي إلى فئة واحدة فقط.
الخيار الوحيد المناسب هو 8D — لأنه يستخدم سمات منطقية متعددة.

Mapping Figure 4.5 using option 8D with Boolean type fields Mflag and Pflag.

ترجمة: تحويل الشكل 4.5 باستخدام الخيار 8D مع سمات نوع منطقية Mflag و Pflag

- الشرح:
لنفترض أن لدينا سوبر كلاس PERSON ، وفئتين فرعيتين متداخلتين:
 - MANAGER (Mflag)
 - PROFESSOR (Pflag)
- العلاقة الناتجة:

```
PERSON (ssn, name, address, Mflag, Pflag, salary, department, office)
```

 - Mflag : TRUE إذا كان الشخص مديرًا.
 - Pflag : TRUE إذا كان شخصًا أستاذًا.
 - يمكن أن يكون كلاهما TRUE (شخص هو مدير وأستاذ في نفس الوقت).
- القيود المطلوبة:
 - لا يمكن أن يكون كلاهما FALSE — لأن الشخص يجب أن يكون على الأقل أحد النوعين.
(CHECK: Mflag OR Pflag = TRUE)
 - إذا كان التخصص منفصلًا، يجب أن يكون: Mflag XOR Pflag = TRUE (أي أحدهما فقط صحيح).
- ملاحظة مهمة:
هذا هو الحل الوحيد المقبول للتصنيفات المتداخلة.
أي محاولة لاستخدام 8C هنا ستؤدي إلى فقدان المعلومات — لأن سمة نوع واحدة لا تستطيع تمثيل انتماءين.

Different Options for Mapping Generalization Hierarchies

ترجمة: الخيارات المختلفة لتحويل هرم التعميم

• الشرح:

هذه الشريحة (9-28) تلخص الخيارات الأربعة السابقة في رسم بياني واحد:

- (a) Option 8A → جدول لكل فئة فرعية + جدول السوبر كلاس
- (b) Option 8B → جدول للسوبر كلاس فقط + جداول للفئات الفرعية تحتوي على السمات الخاصة فقط
- (c) Option 8C → جدول واحد مع سمة نوع واحدة
- (d) Option 8D → جدول واحد مع سمات نوع منطقية متعددة

• ملاحظة مهمة:

الخيار الأمثل يعتمد على:

- هل التخصص منفصل أم متداخل؟
- هل التخصص معرف بالسمة أم لا؟
- ما مدى تكرار الاستعلامات؟ (هل نريد استرجاع كل الأنواع؟ أم نوع معين؟)
- هل الأداء أهم أم التخزين؟

Fig. 9.5: Different Options for Mapping Generalization Hierarchies - summary

ترجمة: الشكل 9.5: ملخص للخيارات المختلفة لتحويل هرم التعميم

• الخلاصة المرئية:

هذه الشريحة تقدم جدولاً مقارنةً بين الخيارات الأربعة، مع الإجابة على الأسئلة التالية:

المعيار	8A	8B	8C	8D
عدد الجداول	N+1	N+1	1	1
دعم التخصص المتداخل	✗	✗	✗	✓
دعم التخصص المعرف بالسمة	✓	✓	✓	✓
قيم NULL	قليلة	قليلة	كثيرة	قليلة (إذا كانت السمات غير اختيارية)
كفاءة الاستعلامات العامة	ضعيفة (JOINS)	ضعيفة (JOINS)	جيدة	جيدة
كفاءة التخزين	متوسطة	ممتازة	متوسطة	متوسطة
سهولة التحديث	جيدة	جيدة	جيدة	جيدة

⚠ ملاحظة حاسمة:

لا يوجد "خيار أفضل مطلقاً" — الاختيار يعتمد على متطلبات النظام.

- إذا كنت تريد أداءً عاليًا واستعلامات سريعة: اختر 8C أو 8D.

- إذا كنت تريد توفير مساحة وتخزين فعال: اختر 8B.
- إذا كنت تريد وضوح هيكل وسهولة الصيانة: اختر 8A.
- إذا كان لديك تداخل: لا خيار سوى 8D.

Mapping of Shared Subclasses (Multiple Inheritance)

ترجمة: تحويل الفئات الفرعية المشتركة (الوراثة المتعددة)

- **التعريف:**
الفئة الفرعية المشتركة (Shared Subclass) هي فئة تُورث من أكثر من سوبر كلاس واحد — وهذا يُعرف بـ الوراثة المتعددة (Multiple Inheritance).
مثال: STUDENT_ASSISTANT ← يورث من STUDENT و EMPLOYEE.
- **شرط أساسي:**
جميع السوبر كلاسات التي تورث منها الفئة المشتركة يجب أن تشترك في نفس المفتاح الأساسي (key attribute).
• لماذا؟ لأننا نريد أن نربط السجل بـ "شخص واحد" — لا يمكن أن يكون له مفتاحان مختلفان.
- **التحويل:**
يمكن تطبيق أي من الخيارات 8A إلى 8D على الفئة المشتركة — ولكن مع قيد:
"يجب أن يكون المفتاح الأساسي موحدًا بين جميع السوبر كلاسات".
- **مثال:**
 - STUDENT(s_id, name, major)
 - EMPLOYEE(e_id, name, salary)
 - إذا كان s_id = e_id (أي نفس الرقم)، فيمكننا عمل:

STUDENT_ASSISTANT (s_id, name, major, salary, assistant_type)

باستخدام Option 8C — لأن المفتاح موحد.

- **ملاحظة مهمة:**
إذا لم يكن المفتاح موحدًا، فلا يمكن تمثيلها كفئة مشتركة — بل يجب استخدام Category (Union Type) — وهو موضوع الشريحة التالية.

FIGURE 4.7 A specialization lattice with multiple inheritance for a UNIVERSITY database.

ترجمة: شبكة تخصص مع وراثة متعددة لقاعدة بيانات جامعة

- **الشرح:**
هذا الشكل يعرض شبكة تخصص (Lattice) حيث:
 - PERSON ← STUDENT و EMPLOYEE
 - STUDENT_ASSISTANT ← STUDENT و EMPLOYEE (وراثة متعددة)

- FACULTY ← EMPLOYEE
- GRADUATE_STUDENT ← STUDENT

• الهدف:

تمثيل الهيكل المعقد حيث بعض الكيانات تملك أكثر من أصل.

• ملاحظة مهمة:

هذا النوع من الهياكل لا يمكن تمثيله بسهولة باستخدام نموذج ER العادي — لكن EER يدعمه، وتحويله يتطلب استخدام Options 8C أو 8D مع التأكد من وحدة المفتاح.

FIGURE 9.6 Mapping the EER specialization lattice in Figure 4.7 using multiple options.

ترجمة: تحويل شبكة التخصص في الشكل 4.7 باستخدام خيارات متعددة

• الشرح:

هنا يتم تطبيق خيارات مختلفة على أجزاء مختلفة من الشبكة:

- STUDENT و EMPLOYEE → تم استخدام Option 8A (جداول منفصلة).
- STUDENT_ASSISTANT → تم استخدام Option 8C (جدول واحد مع type = "StudentAssistant").
- السمات المشتركة (مثل: ssn, name) موروثة من PERSON.

• النتيجة:

• 4 جداول: PERSON, STUDENT, EMPLOYEE, STUDENT_ASSISTANT

- STUDENT_ASSISTANT تحتوي على:
ssn, name, student_id, employee_id, type
(كمفتاح ssn فهي تستخدم، student_id = employee_id = ssn لكن بما أن).

• ملاحظة مهمة:

هذا المزيج من الخيارات (Hybrid Approach) شائع في التطبيقات الواقعية — حيث لا نلتزم بخيار واحد فقط، بل نختار الأنسب لكل جزء.

Step 9: Mapping of Union Types (Categories)

ترجمة: الخطوة 9: تحويل أنواع الاتحاد (الفئات)

• التعريف:

الفئة (Category) أو نوع الاتحاد (Union Type) هي مجموعة من الكيانات من سوپر كلاسات مختلفة (أي ليسوا في علاقة تعميم مباشرة)، ونريد تجميعهم في كيان واحد.

- مثال: OWNER — يمكن أن يكون شخصًا (PERSON) أو شركة (COMPANY) — وهما ليسا مرتبطتين بترابط تعميم.

• التحدي:

السوبر كلاسات لديها مفاتيح أساسية مختلفة — لا يمكن توحيد المفتاح.

• **الحل:**

نستخدم مفتاح اصطناعي (Surrogate Key) — وهو مفتاح لا يحمل معنى منطقي، ويُنشأ فقط لتمثيل الفئة.

• مثال: OwnerId (رقم تسلسلي) — مفتاح أساسي لجدول OWNER.

• **العلاقة الناتجة:**

OWNER (OwnerId, owner_type, owner_id_value)

- owner_type : "Person" أو "Company"
- owner_id_value : إذا كانت CompanyID (إذا كان شخصًا، أو SSN) قيمة المفتاح من الجدول الأصلي : (شركة)

• **الروابط الخارجية:**

- لكن هذا غير — COMPANY.company_id أو PERSON.ssn يشير إلى OWNER.owner_id_value (لأنه يشير إلى جدولين مختلفين) SQL مدعوم مباشرة في
- لحل هذا، نستخدم قيود تطبيقية (Application-Level Constraints) أو جداول مرتبطة منفصلة.

• **ملاحظة مهمة:**

الفئة (Category) ليست تخصصًا — لأن التخصص يكون بين كيانات من نفس "العائلة" (تعميم)، أما الفئة فهي تجمع كيانات من عائلات مختلفة.

FIGURE 4.8 Two categories (union types): OWNER and REGISTERED_VEHICLE.

ترجمة: فنتان (أنواع اتحاد): المالك والمركبة المسجلة

• **الشرح:**

- OWNER : يمكن أن يكون PERSON أو COMPANY
- REGISTERED_VEHICLE : يمكن أن يكون CAR أو TRUCK

هذان مثالان على فئات اتحاد — لأن كلاهما يجمع كيانات من أصول مختلفة.

• **الهدف من الفئة:**

تمكين عملية مثل: "أظهر جميع مالكي السيارات" — حتى لو كان المالك شخصًا أو شركة.

FIGURE 9.7 Mapping the EER categories (union types) in Figure 4.8 to relations.

ترجمة: تحويل فئات EER في الشكل 4.8 إلى علاقات

• **العلاقات الناتجة:**

1. PERSON(ssn, name, address)
2. COMPANY(company_id, name, address)
3. CAR(car_id, model, year)

4. TRUCK(truck_id, capacity)
5. OWNER(owner_id, owner_type, owner_ref)
 - owner_type : 'Person' أو 'Company'
 - owner_ref : ssn (إذا كان person) أو company_id (إذا كانت company)
6. REGISTERED_VEHICLE(reg_id, vehicle_type, vehicle_ref)
 - vehicle_type : 'Car' أو 'Truck'
 - vehicle_ref : car_id أو truck_id

• القيود المطلوبة:

- لا يمكن أن يكون owner_ref فارغاً.
- يجب أن يتطابق owner_type مع نوع owner_ref — أي: إذا كان owner_type='Person' ، يجب أن يكون owner_ref موجوداً في PERSON .
- هذه القيود لا يمكن تنفيذها تلقائياً في SQL — يجب كتابتها ك Triggers أو في مستوى التطبيق.

• ملاحظة مهمة:

استخدام المفتاح الاصطناعي (owner_id) هو الحل الوحيد المقبول للفئات — لأنه يحل مشكلة اختلاف المفاتيح.

Mapping Exercise-1: Exercise 9.4: Map this schema into a set of relations. FIGURE 9.8 An ER schema for a SHIP_TRACKING database.

ترجمة: تمارين التحويل 1: التمرين 9.4: حوّل هذا المخطط إلى مجموعة من العلاقات. الشكل 9.8: مخطط ER لقاعدة بيانات تتبع السفن

(ملاحظة: بما أن الشكل غير مرفق، سنفترض أنه يحتوي على علاقات بسيطة) لكن القاعدة العامة:

- تطبيق خطوات 1 إلى 7 (التحول من ER إلى علاقات) ثم خطوة 8 إذا كان هناك تخصص.
- إذا كان هناك تخصص منفصل، استخدم 8A أو 8B.
- إذا كان هناك فئة، استخدم Surrogate Key.
- إذا كان هناك تخصص متداخل، استخدم 8D.

Mapping Exercise-2: Exercise 9.9: Map this schema into a set of relations. FIGURE 9.9 EER diagram for a car dealer

ترجمة: تمارين التحويل 2: التمرين 9.9: حوّل هذا المخطط إلى مجموعة من العلاقات. الشكل 9.9: مخطط EER لوكيل سيارات

افتراض عام:

- قد يحتوي على: $VEHICLE \leftarrow CAR, TRUCK, MOTORCYCLE$
- وقد يكون هناك $SALE$ و $CUSTOMER$
- وقد يكون هناك فئة مثل $CUSTOMER \leftarrow BUYER$ أو $COMPANY$

الخطوات المقترحة:

1. حوّل الكيانات العادية (Regular Entities) إلى جداول.
 2. حوّل العلاقات $N:1$ إلى مفاتيح خارجية.
 3. حوّل العلاقات $M:N$ إلى جداول منفصلة.
 4. إذا كان هناك تخصص في $VEHICLE$ ، فاختر:
 5. إذا كان هناك فئة مثل $BUYER$ من $CUSTOMER$ و $DEALER$ ، استخدم **Surrogate Key**.
- إذا أردت وضوحاً 8A.
 - إذا أردت أداءً 8C.
 - إذا كان هناك تداخل (مثلاً: سيارة كهربائية ورياضية في نفس الوقت) 8D.

Chapter Summary

ترجمة: ملخص الفصل

ER-to-Relational Mapping Algorithm

(خوارزمية تحويل نموذج ER إلى علاقات)

الخطوة	الوصف
Step 1	تحويل الكيانات العادية (Regular Entity Types) إلى جداول — كل كيان = جدول، وكل سمة = عمود، والمفتاح الأساسي يصبح PK.
Step 2	تحويل الكيانات الضعيفة (Weak Entities) — تُنشئ جدولاً يحتوي على سماته + المفتاح الخارجي للكيان المميز (Owner Entity).
Step 3	تحويل العلاقات الثنائية $1:1$ — يمكن وضع المفتاح الخارجي في أي من الجدولين (أو كليهما).
Step 4	تحويل العلاقات الثنائية $N:1$ — المفتاح الخارجي يوضع في الجانب "N".
Step 5	تحويل العلاقات الثنائية $M:N$ — تُنشئ جدولاً منفصلاً يحتوي على مفتاحين خارجيين (من كلا الطرفين) + أي سمات للعلاقة.
Step 6	تحويل السمات متعددة القيم (Multivalued Attributes) — تُنشئ جدولاً منفصلاً يحتوي على المفتاح الخارجي + السمة.
Step 7	تحويل العلاقات N -ary — تُنشئ جدولاً منفصلاً يحتوي على مفاتيح خارجية من جميع الكيانات المشاركة.

Mapping EER Model Constructs to Relations

الخطوة	الوصف
Step 8	تحويل التخصص والتعميم (Specialization/Generalization) باستخدام أحد الخيارات الأربعة: 8A, 8B, 8C, 8D — حسب نوع التخصص (منفصل/متداخل/محدد بالسمة).
Step 9	تحويل الفئات (Categories/Union Types) باستخدام مفتاح اصطناعي (Surrogate Key) لأن السوبر كلاسات مختلفة المفاتيح.

✓ **ملخص نهائي شامل — ملاحظات حاسمة

المفهوم	الخلاصة	ما يجب تذكره
Option 8A	جدول لكل فئة فرعية + جدول السوبر كلاس	الأسهل للفهم، الأصعب للأداء. مناسب للأنظمة التي تحتاج وضوحًا.
Option 8B	جدول السوبر كلاس + جداول الفئات الفرعية تحتوي على السمات الخاصة فقط	أفضل من حيث التخزين، أسوأ من حيث الأداء (كثير من JOINS).
Option 8C	جدول واحد + سمة نوع واحدة	مناسب للتخصص المعرف بالسمة والمنفصل. لا يدعم التداخل.
Option 8D	جدول واحد + سمات منطقية متعددة	الوحيد الذي يدعم التخصص المتداخل. مثالي عندما يكون الكائن قد يكون من نوعين.
Shared Subclass	فئة تورث من أكثر من سوبر كلاس	شرط أساسي: جميع السوبر كلاسات يجب أن تشترك في نفس المفتاح.
Category (Union Type)	تجمع كيانات من سوبر كلاسات مختلفة	يجب استخدام مفتاح اصطناعي (Surrogate Key) — لأنه لا يوجد مفتاح مشترك.
الفرق بين Specialization و Category	التخصص: كيانات من نفس "العائلة" (تعميم). الفئة: كيانات من عائلات مختلفة.	لا تخط بينهما! التخصص = تعميم، الفئة = اتحاد.
القيود في 8D و Categories	لا يمكن تنفيذها تلقائيًا في SQL	تحتاج إلى Triggers أو تطبيق (Application Logic).
المفتاح الاصطناعي (Surrogate Key)	رقم تسلسلي (مثل: ID) لا معنى له منطقيًا	يستخدم فقط في الفئات (Categories)، وليس في التخصصات.

📌 نصائح ذهبية :

- إذا رأيت "disjoint" → لا تستخدم 8D، استخدم 8A أو 8C.
- إذا رأيت "8D" → "overlapping" هو الحل الوحيد.

3. إذا رأيت "8C" → "attribute-defined" هو الأنسب.
4. إذا رأيت شخصاً وشركة كمالكين → فئة (Category) → surrogate key.
5. إذا رأيت "STUDENT" ← "STUDENT_ASSISTANT" و "EMPLOYEE" → shared subclass
تحقق من وحدة المفتاح!
6. لا تنس: في 8B، جدول السوبر كلاس لا يحتوي على سمات الفئات الفرعية — فقط السمات المشتركة.
7. في 8A، كل جدول فرعي يحتوي على كل سمات السوبر كلاس + سماته الخاصة — أي تكرار.
8. الاختيار الأمثل يعتمد على: التخصص، التداخل، الأداء، التخزين — لا يوجد "أفضل" مطلق.